Addis Ababa University

Master's in Artificial Intelligence

Computer Vision (CV) Laboratory Manual

Prepared by: Dr. Natnael Argaw Wondimu

Environment: OpenCV and Visual Studio

Preface

This manual is prepared in a workbook style for AI Master's students at Addis Ababa University. It provides a hands-on introduction to Computer vision with OpenCV, covering advanced topics. Each lab includes objectives, theoretical background, procedures, OpenCV code, checkpoints, try-it-yourself prompts, and collaborative assignments. The manual is intended to serve as a base for advanced studies in Computer Vision.

Chapter 1: Introduction to Computer Vision

Objective

To introduce the basic concepts of computer vision, its historical development, and practical setup using OpenCV. Students will perform simple image I/O operations, visualize color spaces, and prepare their environment for more advanced labs.

1. What is Computer Vision?

Description: Computer Vision is a field of AI that enables machines to interpret and understand the visual world by extracting useful information from digital images and video.

2. Brief History & Applications

2.1 History

- 1960s: Beginnings in pattern recognition and robotics vision.
- 1980s–90s: Rise of image analysis and mathematical modeling.
- 2010s–present: Deep learning revolutionizes the field.

2.2 Real-World Applications

- Face detection and recognition (e.g., phone unlock)
- Autonomous vehicles (lane and obstacle detection)
- Industrial inspection (defect detection in manufacturing)
- Augmented Reality (object tracking, pose estimation)
- Medical imaging (tumor segmentation, X-ray analysis)

3. Environment Setup

3.1 Installing Python and OpenCV

Steps: 1. Install Python from https://www.python.org 2. Install OpenCV via pip:

```
pip install opencv-python
pip install opencv-python-headless  # for headless environments
```

3.2 Verifying Installation

```
import cv2
print(cv2.__version__)
```

Expected Output: The installed OpenCV version number (e.g., '4.9.0').

4. Image I/O with OpenCV

4.1 Reading and Displaying an Image

```
import cv2
img = cv2.imread('sample.jpg')
cv2.imshow('Image', img)
```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output Description: Opens a window displaying the image. Press any key to close.

4.2 Writing (Saving) an Image

```
cv2.imwrite('saved_sample.jpg', img)
```

Output Description: Saves a copy of the displayed image to the working directory.

5. Color Spaces

5.1 What Are Color Spaces?

Description: Different representations of color. RGB is most common, but other spaces like HSV and Grayscale are useful for various tasks.

5.2 Convert BGR to Grayscale

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
cv2.imshow('Grayscale', gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output Description: Displays a grayscale version of the image.

5.3 Convert BGR to HSV

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
cv2.imshow('HSV Image', hsv)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Output Description: Displays the HSV-transformed image. Useful for color-based segmentation.

6. Summary

- OpenCV allows reading, writing, and transforming images efficiently.
- Grayscale is ideal for edge and intensity processing.
- HSV is helpful in color detection and filtering.

Suggested Exercises

1. Load and display your own image.
2. Save the image in PNG and BMP formats.
3. Try converting the image to other color spaces (e.g., LAB, YCrCb).
4. Use cv2.split() to view individual RGB/H channels.